



ASSIGNMENT 3

Teacher: Sir Bilal Khan

COURSE: Object Oriented Programming

SECTION: SE-2A, CS-2A, CS-2B, CS-2C

Instructions:

All assignments should be done individually.

Plagiarized work will be given zero marks.

Make sure your code compiles and runs.

Submit your .cpp files. Name your files as

(P25-1234_Name_Section.cpp)

Submit PDF Files with outputs as well.

Late submissions will not be entertained.

If there is any confusion make assumptions or ask the TA.

Rules:

- No vectors, lists, or maps
- No fixed-size arrays
- Must resize arrays manually
- Must make sure proper memory deallocation
- Classes and Objects
- Constructors

UNIVERSITY MANAGEMENT SYSTEM

Design a menu-driven C++ program using:

- Static Keyword
- Const Keyword
- Inheritance (public, protected, private)
- Runtime Polymorphism

System Hierarchy (Using Classes):

- University
 - Departments (Dynamic Array)
 - Persons (Dynamic Array of base class pointer)
 - Students
 - Teachers
 - Staff

----- Class Design -----

Person Class (Base Class):

- string name
- int id

Functions:

- Virtual displayInfo() const
- Virtual destructor

Student Class (Public Inheritance):

- string course
- float marks

Teacher Class (Protected Inheritance):

- string subject
- int experience

Staff Class (Private Inheritance):

- string department

Department Class:

- string deptName
- Person ** persons
- int personCount

University Class:

- string uniName
- Department * depts
- int deptCount
- static int totalPersons

Requirements:

- Use default constructors to initialize values
- Use parameterized constructors for assigning data
- Use static variable to track total persons
- Use const functions where modification is not required
- All arrays must be dynamically allocated
- Must demonstrate:
 - Public inheritance (Student)
 - Protected inheritance (Teacher)
 - Private inheritance (Staff)
- Must use base class pointer (Person*) for polymorphism

Input Policy:

User enters number of departments

For each department:

- Enter department name

Add persons:

- Select department
- Choose type (Student / Teacher / Staff)

For each type:

- Student → name, id, course, marks
- Teacher → name, id, subject, experience
- Staff → name, id, department

i

Menu Example and Functions (Must be separate functions):

1. Add Department:

- Resize department array (+1)
- Take department details

2. Add Person:

- Select department
- Choose type (Student / Teacher / Staff)
- Create object using parameterized constructor
- Store using Person*
- Resize persons array

3. Display University Data (use const function):

Display:

- University name
- All departments
- All persons inside each department
- Use polymorphism (displayInfo())

4. Show All Teachers:

- Traverse all departments
- Call wrapper function (due to protected inheritance)

5. Remove Person:

- Select department → person
- Delete object
- Decrease static count
- Resize array manually
- Properly deallocate memory

6. Show Total Persons:

- Display total persons using static variable

7. Exiting the Program:

Before exiting:

- Deallocate all persons
- Deallocate all departments

SAMPLE MENU:

```
===== UNIVERSITY MANAGEMENT SYSTEM =====

1. Add Department
2. Add Person
3. Display University Data
4. Show All Teachers
5. Remove Person
6. Total Persons
7. Exit

Enter your choice:
=====
```

SMART TRANSPORT MANAGEMENT SYSTEM

You must implement a class hierarchy where **Vehicle** is the base class and **Bus**, **Car**, and **Bike** are derived using different types of inheritance. All vehicles should be stored using **base class pointers** to demonstrate **runtime polymorphism**. The program should support adding routes, adding/removing vehicles, displaying all data, and tracking total vehicles using a **static variable**.

System Hierarchy

- Routes (Dynamic Array)
 - Vehicles (Dynamic Array of base class pointer)
 - Bus
 - Car
 - Bike

----- Class Design -----

Base Class: Vehicle

- String name
- int vehicleID

Functions:

- virtual void displayInfo() const
- virtual ~Vehicle()

----- Derived Classes -----

Bus (Public Inheritance)

- int capacity
- string route_name

Car (Protected Inheritance)

- string model
- float fare_per_Km

Bike (Private Inheritance)

- int engineCC

----- Route Class -----

- string Route_name
- Vehicle** vehicles
- int Vehicle_count

----- Transport System Class -----

- string systemName
- Route* routes
- int route_count
- static int total_vehicles

Functional Requirements (Menu Options):

Must use **runtime polymorphism**

1. Add Route
2. Add Vehicle
3. Display All Data (const function)
4. Show All Cars
5. Remove Vehicle
6. Show Total Vehicles
7. Exit Program

SAMPLE MENU:

```
--- TRANSPORT MANAGEMENT SYSTEM ---  
  
1. Add Route  
2. Add Vehicle  
3. Display  
4. Remove Vehicle  
5. Total Vehicles  
0. Exit  
Choice: 
```
